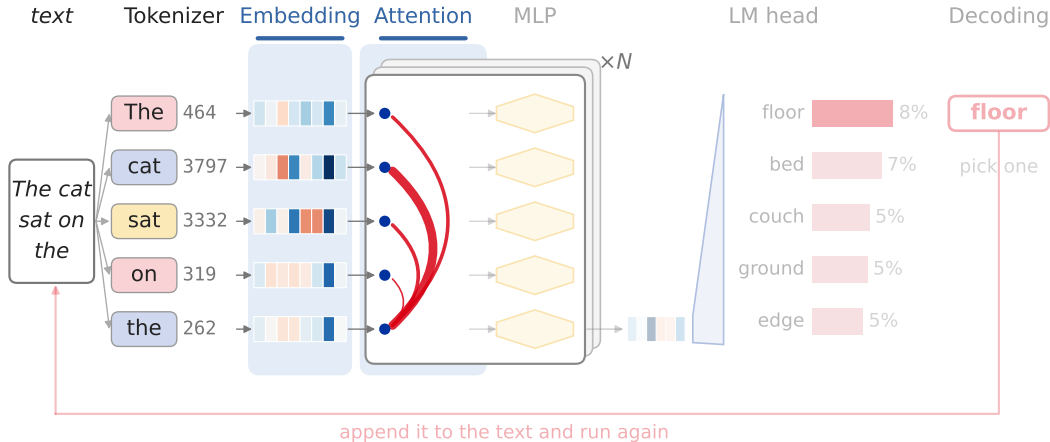


Lecture 24: Attention

How tokens share meaning

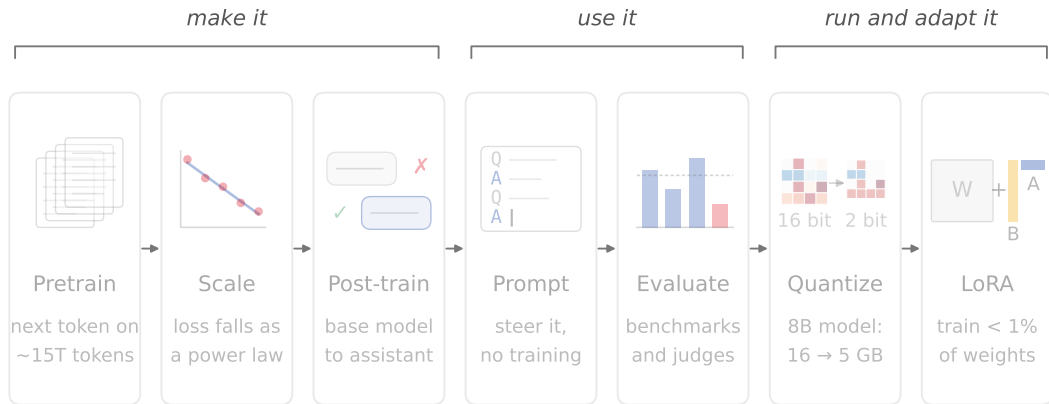
LLM chapter. Parts adapt LMU I2DL (CC BY 4.0) and 3Blue1Brown, “Deep Learning” Ch. 5–6.

Where we are: one forward pass



Today: the **embedding** and the **attention** step - how each token's vector picks up meaning from the other tokens.

Where we are: the life of a model



Later: where these weights come from, and what you do with them once you have them.

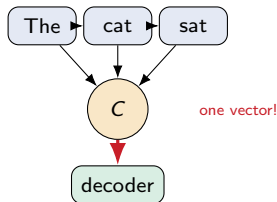
Last chapter ended on a cliffhanger

In L21, the encoder read a whole sentence and crushed it into **one** context vector C . The decoder then wrote the translation from C alone.

For a short sentence, fine. For a long one, C is a bottleneck – the ending forgets the beginning.

Even the LSTM could not fix it: the memory is still one fixed vector.

The question we left open: *what if the decoder could look back at **all** the encoder states, not just the last one?*



The fix, in one idea

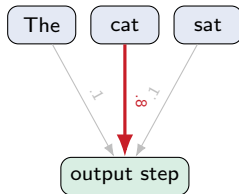
Predict: how would *you* fix the bottleneck?

The fix, in one idea

Predict: how would *you* fix the bottleneck?

Don't force everything through one vector.
At **each** step of writing the output, let the decoder **look back over every input state** and take a *weighted average* – heavier weight on the states that matter right now.

That weighted look-back *is* **attention**. The whole chapter unpacks this one sentence – and then takes it much further.



weights sum to 1; here the output leans on “cat”

Words are vectors; directions carry meaning

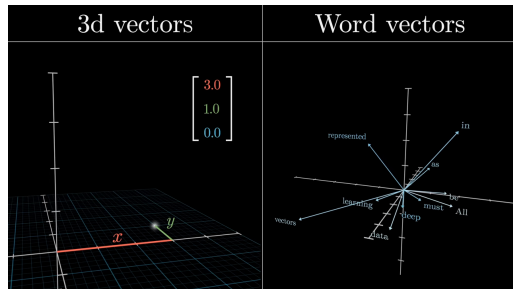
Attention moves a word's vector around a meaning-space. First, recall what that space is.

Recall: a token is a vector

Every token is mapped to a vector – its **embedding** – a point in a high-dimensional space (GPT-3: 12,288 dims). The lookup itself is not new: it is the row-picking one-hot $\times \mathbf{W}$ product from L20's anatomy frame, made big and learned.

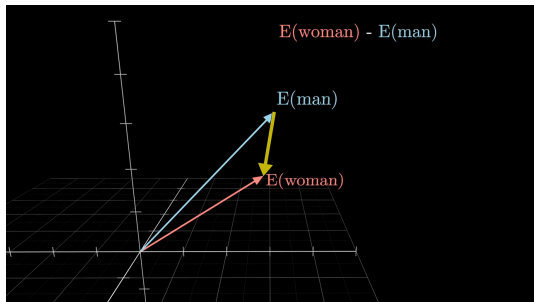
The key idea: **directions in this space carry meaning**. Nearby vectors are similar words.

A word starts as a lookup – *context-free*. The whole job of the network is to enrich it.



3Blue1Brown

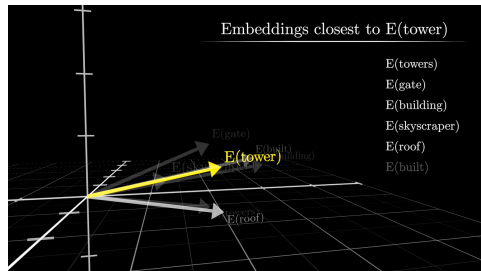
Directions carry meaning



The famous one: $\text{king} - \text{man} + \text{woman} \approx \text{queen}$. A single *direction* encodes “gender”; another “royalty.” (word2vec:

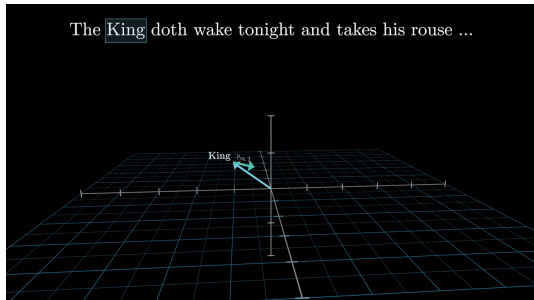
Mikolov et al., 2013)

3Blue1Brown



The **dot product** measures how much two vectors align. **Remember this** – it is the engine of attention.

Embeddings soak up context



3Blue1Brown

The same word means different things: “a machine learning **model**” vs. “a fashion **model**.” The starting embedding of *model* is identical in both.

Something has to **pull** that generic vector toward the right meaning, using the surrounding words.

That pull is exactly what attention does: move each embedding to a context-specific point in meaning-space.

"Each of you has what on the surface appears an unassailable alibi." she said, as the storm cleared and the crimson rays of dusk poured through the window where everyone was gathered. "But only one of you could have known about the loose screw on the window, while also having known where the second key was hidden. I am left, inescapably then, to the conclusion that **therefore, the murderer was** ???


$$\begin{bmatrix} -2.7 \\ -2.4 \\ +8.8 \\ -4.7 \\ -0.4 \\ +6.1 \\ +6.2 \\ +4.8 \end{bmatrix}$$

To fill the blank, the last vector alone has to have soaked up the whole context.

$$\begin{bmatrix} \vdots \\ -6.2 \end{bmatrix}$$

The attention idea

Score, softmax, weighted sum – the whole mechanism in three moves.

Attention = a weighted look-back

At each output position, attention does three things:

1. **Score** every other position for relevance (how much should I look there?).
2. **Softmax** the scores into weights that are positive and sum to 1.
3. **Weighted sum**: blend the other positions' information by those weights.

That's it – a **content-based average**. The rest of this deck is just *how* we compute the scores and *what* we blend. No new calculus: softmax and weighted sums are L14.

You can see it

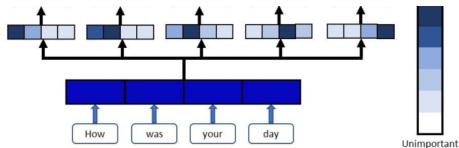


Figure: An illustration of a machine translation task using an encoder-decoder model with an attention mechanism. The attention weights at each time-step of the decoding/translation process indicate which parts of the input sequence are most relevant. There are 4 attention weights because there are 4 encoder states (Loye, 2019).



A woman is throwing a frisbee in a park.



A stop sign is on a road with a mountain in the background.

image captioning: the attention mechanism tells the

Translation: each output word's attention over the input words
– the model learns the alignment itself. (Bahdanau, Cho & Bengio, 2015 – the first attention)

Captioning: attention over *pixels* while writing each word. (Xu et al., 2015)

Same skeleton – score, softmax, weighted sum – whether the “positions” are source words or image patches. Next: let the model *learn* what to score.

The leap: drop the RNN

If attention gives direct access to everything, the sequential RNN is dead weight.

Why keep the RNN at all?

Predict: attention already lets any output position pull from any input position, *directly*.
So why keep the slow, sequential RNN underneath – the part that forces us to process one token at a time?

Why keep the RNN at all?

Predict: attention already lets any output position pull from any input position, *directly*. So why keep the slow, sequential RNN underneath – the part that forces us to process one token at a time?

You don't. Rip out the recurrence, keep only attention. That is the 2017 paper whose title says it all: **“Attention Is All You Need.”**

(Vaswani et al., 2017)

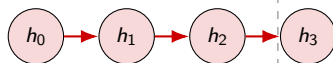
Stack these attention layers and the vectors get refined again and again – meaning deepens with depth.



3Blue1Brown

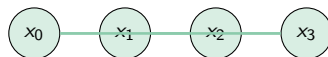
Sequential vs. all-to-all

RNN / LSTM



sequential: $O(n)$ steps to connect ends

Transformer



all-to-all: $O(1)$ – direct

Three wins: **parallel** (all tokens at once) · **direct access** (any token to any other) · **contextual** (representations change with context).

Why context matters

American shrew mole

One mole of carbon dioxide

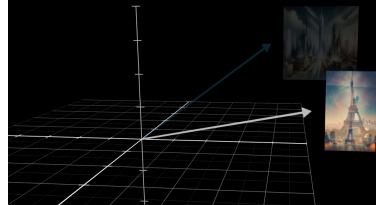
Take a biopsy of the mole

mole means three different things – but its starting embedding is identical in all three.

Self-attention is how each word pulls the context it needs from the *same* sentence. Let's build it – slowly.

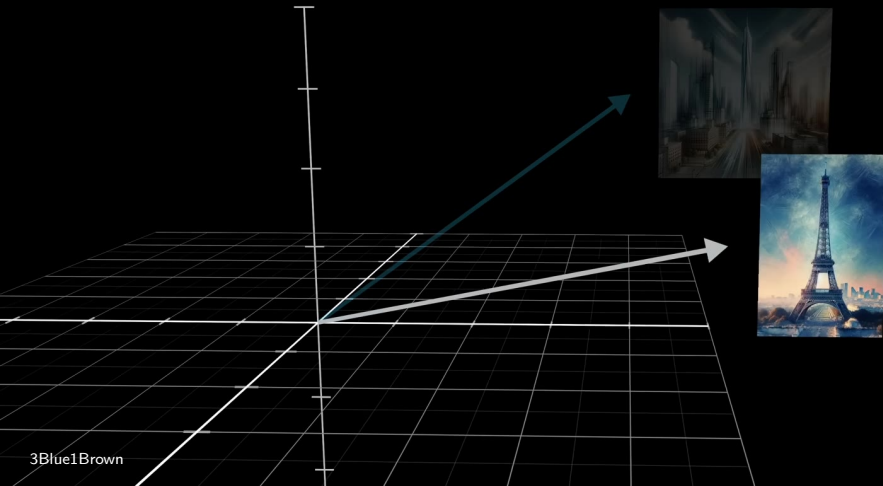
3Blue1Brown

Eiffel Tower



tower, once preceded by Eiffel, should move toward Paris/France/steel.

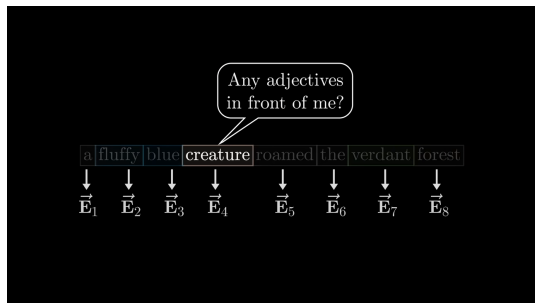
Eiffel Tower



Self-attention: query, key, value

The chapter's one deep dive. We go slow – what each piece *is*, and *why* the equation looks the way it does.

The goal, made concrete



3Blue1Brown

Sentence: *"a fluffy blue creature roamed the verdant forest."*

We want the noun **creature** to absorb its adjectives **fluffy** and **blue**, so its vector means *"a fluffy, blue creature."*

Two questions the mechanism must answer:

- ▶ Which words are **relevant** to creature?
- ▶ **What** should they contribute?

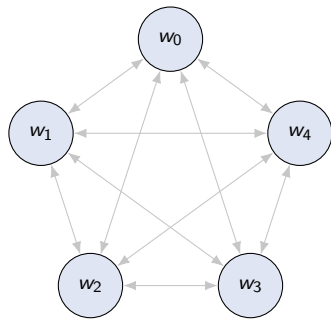
"Each of you has what on the surface appears an unassailable alibi." she said, as the storm cleared and the crimson rays of dusk poured through the window where everyone was gathered. "But only one of you could have known about the loose screw on the window, while also having known where the second key was hidden. I am left, inescapably then, to the conclusion that therefore, the murderer was ???


$$\begin{bmatrix} -1.9 \\ -3.0 \\ +8.2 \\ -5.3 \\ +1.1 \\ +5.0 \\ +4.6 \\ +2.8 \\ -3.7 \\ +0.5 \\ \vdots \\ -6.5 \end{bmatrix}$$

Why “self”-attention? The sentence talks to itself

In the translation attention from the start of this lecture, queries came from the *decoder* and keys/values from the *encoder* – two different sequences.

In **self**-attention, Q , K and V all come from the **same** sentence. Every word at once **asks** (its query) and **answers** (its key and value). One round is **all-to-all messaging**: each word gathers what it needs from every other word.



every word attends to every word

Think of it as **routing**: attention decides who talks to whom, and the values are the messages passed.

The analogy: a soft dictionary lookup

Think of looking something up in a library:

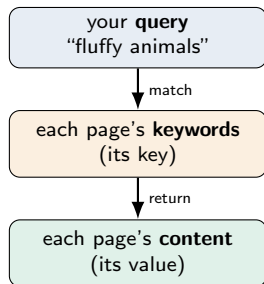
- ▶ You arrive with a **query** – what you want.
- ▶ Each book has a **key** – its label on the spine.
- ▶ Each book has a **value** – its actual contents.

You compare your query to *every* key, then take the **values** of the good matches.

A normal dictionary is a **hard** lookup: one key matches exactly. Attention is a **soft** lookup: *every* key matches a little, weighted by similarity – and you blend all the values.

Every word will produce its own query, key, and value. Next: where these come from.

Another way to picture it: web search

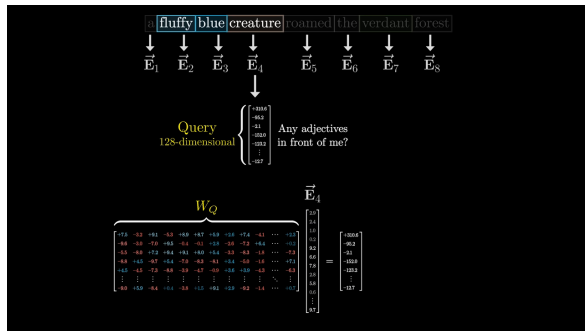


A search engine compares your **query** to every page's **keywords**, ranks the matches, and returns the best pages' **content**.

Attention is the same idea – with one twist: it doesn't return only the top hit. It returns a **blend** of every value, weighted by how well each key matched.

Query = what you search for · **Key** = how a word is indexed · **Value** = what it hands back.

Query — “what am I looking for?”



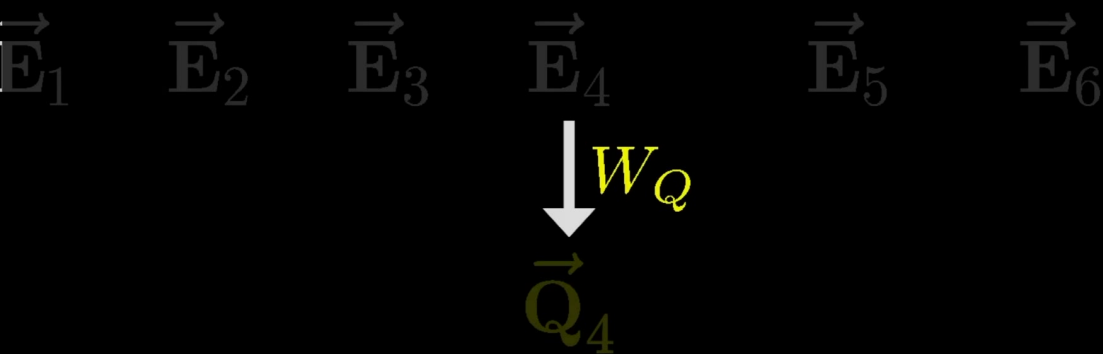
Multiply a word's embedding by a learned matrix W_Q to get its **query** vector (small – say 128-dim):

$$\mathbf{q}_i = W_Q \mathbf{e}_i.$$

Think of *creature*'s query as encoding the question “*are there any adjectives in front of me?*”

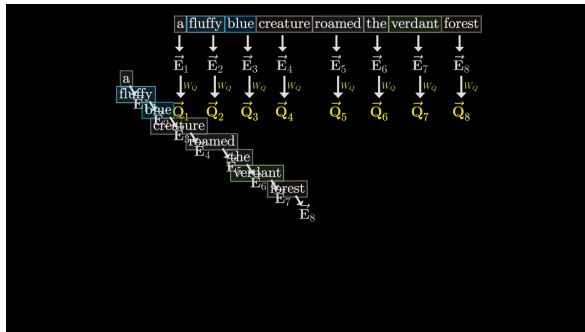
W_Q is **learned**. What each word “asks” is discovered from data, not written by us.

3Blue1Brown



Any adjectives
in front of me?

Key — “what do I offer?”



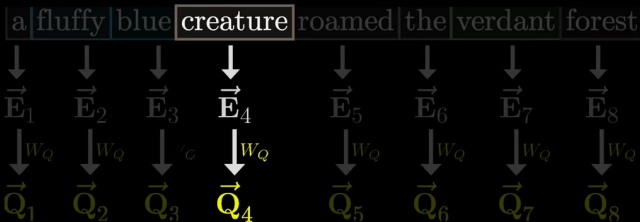
A second learned matrix W_K turns each embedding into a **key**:

$$\mathbf{k}_j = W_K \mathbf{e}_j.$$

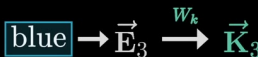
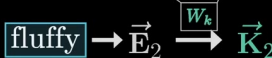
A key *answers* queries. We'd want *fluffy's* key to encode “*I am an adjective, sitting just here.*”

A key is a word *advertising* itself – so that the right queries can find it.

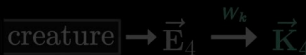
3Blue1Brown



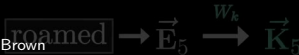
I'm an adjective!
I'm there!



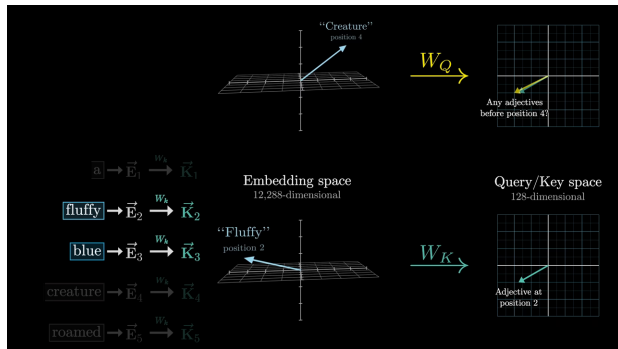
Any adjectives
in front of me?



I'm an adjective!
I'm there!



Queries and keys meet in one space



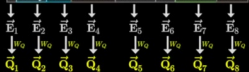
3Blue1Brown

W_Q and W_K **project** the big embedding space down to a shared, smaller **query/key space**.

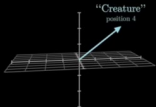
A good W_Q, W_K arrange it so that *creature*'s query ("adjectives before me?") and *fluffy*'s key ("adjective, here") point the **same way** – so they align.

That is *all* W_Q, W_K do: learn a space where "relevant" means "aligned."

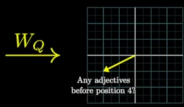
a fluffy blue creature roamed the verdant forest



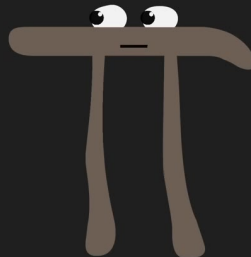
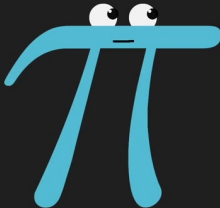
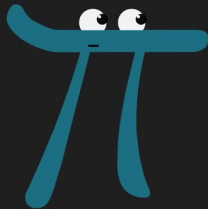
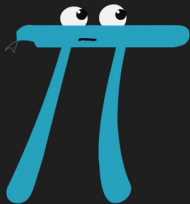
Embedding space
12,288-dimensional



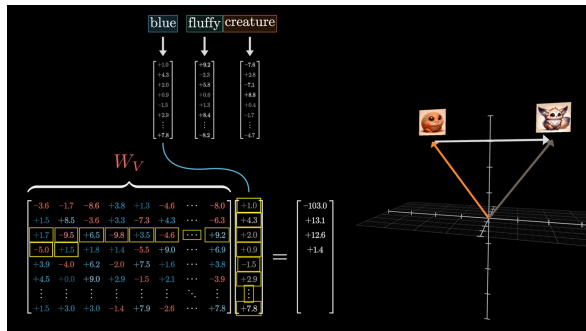
Query/Key space
128-dimensional



What does W_Q do
to the non-nouns?



Value — “what do I pass on?”



A third learned matrix W_V makes each word's **value** – the information it will hand over if attended to:

$$\mathbf{v}_j = W_V \mathbf{e}_j.$$

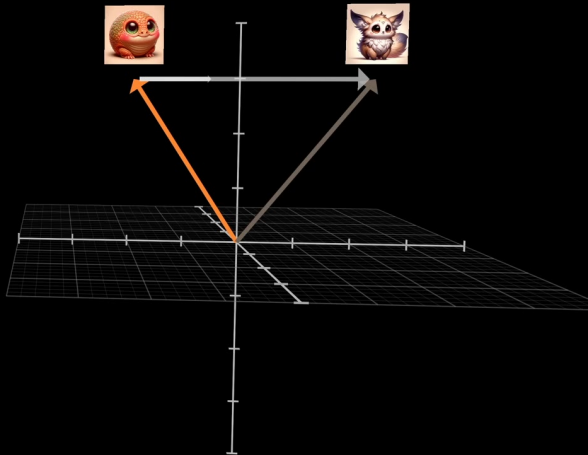
fluffy's value is, roughly, “the fluffiness to add to a noun.”

Key = how you're *found*; value = what you *give*. They are different jobs, so they get different matrices.

3Blue1Brown

fluffy creature


$$\begin{bmatrix} +9.2 \\ -2.3 \\ +5.8 \\ +0.6 \\ +1.3 \\ +8.4 \\ \vdots \\ -8.2 \end{bmatrix}$$

$$\begin{bmatrix} -7.6 \\ +2.8 \\ -6.9 \\ +8.8 \\ +0.6 \\ -1.6 \\ \vdots \\ -4.5 \end{bmatrix}$$


Why three separate matrices?

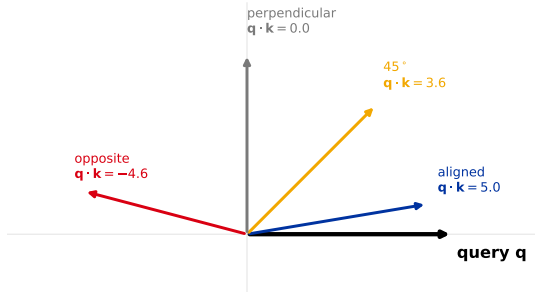
One word plays **three different roles** at once:

Query $\mathbf{q} = W_Q \mathbf{e}$	what I'm <i>looking for</i>	(creature: "adjectives?")
Key $\mathbf{k} = W_K \mathbf{e}$	what I <i>advertise</i>	(fluffy: "I'm an adjective")
Value $\mathbf{v} = W_V \mathbf{e}$	what I <i>contribute</i>	(fluffy: "fluffiness")

Why not just dot the embeddings together directly? Because "*what I'm looking for*" is not the same as "*what I am*" is not the same as "*what I'll give*." Three matrices let the model learn the three roles **independently** – and they are the *only* new weights in attention.

The dot product, up close

Dot product = alignment (big when they point the same way)



The dot product $\mathbf{q} \cdot \mathbf{k} = |\mathbf{q}| |\mathbf{k}| \cos \theta$ is **large** when two vectors point the same way, **zero** when perpendicular, and **negative** when opposite.

So the whole question “how relevant is this key to my query?” collapses to **one number**. Aligned = relevant.

This single operation is the heart of attention. Everything else – softmax, the weighted sum – is bookkeeping around it.

Matching: the dot product scores relevance

	a	fluffy	blue	creature	roamed	the
	$\downarrow \vec{E}_1$	$\downarrow \vec{E}_2$	$\downarrow \vec{E}_3$	$\downarrow \vec{E}_4$	$\downarrow \vec{E}_5$	$\downarrow \vec{E}_6$
	$\downarrow w_Q$	$\downarrow w_Q$	$\downarrow w_Q$	$\downarrow w_Q$	$\downarrow w_Q$	$\downarrow w_Q$
	$\vec{Q}_1$	$\vec{Q}_2$	$\vec{Q}_3$	$\vec{Q}_4$	$\vec{Q}_5$	$\vec{Q}_6$
$\boxed{\text{a}} \rightarrow \vec{E}_1 \xrightarrow{w_k} \vec{K}_1$	$\vec{K}_1 \cdot \vec{Q}_1$	$\vec{K}_1 \cdot \vec{Q}_2$	$\vec{K}_1 \cdot \vec{Q}_3$	$\vec{K}_1 \cdot \vec{Q}_4$	$\vec{K}_1 \cdot \vec{Q}_5$	$\vec{K}_1 \cdot \vec{Q}_6$
$\boxed{\text{fluffy}} \rightarrow \vec{E}_2 \xrightarrow{w_k} \vec{K}_2$	$\vec{K}_2 \cdot \vec{Q}_1$	$\vec{K}_2 \cdot \vec{Q}_2$	$\vec{K}_2 \cdot \vec{Q}_3$	$\vec{K}_2 \cdot \vec{Q}_4$	$\vec{K}_2 \cdot \vec{Q}_5$	$\vec{K}_2 \cdot \vec{Q}_6$
$\boxed{\text{blue}} \rightarrow \vec{E}_3 \xrightarrow{w_k} \vec{K}_3$	$\vec{K}_3 \cdot \vec{Q}_1$	$\vec{K}_3 \cdot \vec{Q}_2$	$\vec{K}_3 \cdot \vec{Q}_3$	$\vec{K}_3 \cdot \vec{Q}_4$	$\vec{K}_3 \cdot \vec{Q}_5$	$\vec{K}_3 \cdot \vec{Q}_6$
$\text{creature} \rightarrow \vec{E}_4 \xrightarrow{w_k} \vec{K}_4$	$\vec{K}_4 \cdot \vec{Q}_1$	$\vec{K}_4 \cdot \vec{Q}_2$	$\vec{K}_4 \cdot \vec{Q}_3$	$\vec{K}_4 \cdot \vec{Q}_4$	$\vec{K}_4 \cdot \vec{Q}_5$	$\vec{K}_4 \cdot \vec{Q}_6$

Score how well key j answers query i with a **dot product** – the same “how-aligned” measure from the embeddings primer:

$$s_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j.$$

Big $\Rightarrow$ “this word is relevant to me.”
Here **fluffy** and **blue** score high against **creature**’s query.

All queries against all keys at once is one matrix product: $S = QK^\top$.

3Blue1Brown

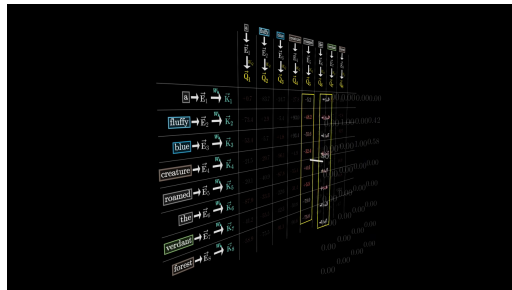
	a	fluffy	blue	creature	roamed	the	ve
	$\downarrow$ $\vec{E}_1$ $\downarrow_{W_Q}$ $\vec{Q}_1$	$\downarrow$ $\vec{E}_2$ $\downarrow_{W_Q}$ $\vec{Q}_2$	$\downarrow$ $\vec{E}_3$ $\downarrow_{W_Q}$ $\vec{Q}_3$	$\downarrow$ $\vec{E}_4$ $\downarrow_{W_Q}$ $\vec{Q}_4$	$\downarrow$ $\vec{E}_5$ $\downarrow_{W_Q}$ $\vec{Q}_5$	$\downarrow$ $\vec{E}_6$ $\downarrow_{W_Q}$ $\vec{Q}_6$	
$\boxed{\text{a}} \rightarrow \vec{E}_1 \xrightarrow{W_k} \vec{K}_1$	$\vec{K}_1 \bullet \vec{Q}_1$	$\vec{K}_1 \bullet \vec{Q}_2$	$\vec{K}_1 \bullet \vec{Q}_3$	$\vec{K}_1 \bullet \vec{Q}_4$	$\vec{K}_1 \bullet \vec{Q}_5$	$\vec{K}_1 \bullet \vec{Q}_6$	$\vec{K}_1 \bullet \vec{Q}_7$
$\boxed{\text{fluffy}} \rightarrow \vec{E}_2 \xrightarrow{W_k} \vec{K}_2$	$\vec{K}_2 \bullet \vec{Q}_1$	$\vec{K}_2 \bullet \vec{Q}_2$	$\vec{K}_2 \bullet \vec{Q}_3$	$\vec{K}_2 \bullet \vec{Q}_4$	$\vec{K}_2 \bullet \vec{Q}_5$	$\vec{K}_2 \bullet \vec{Q}_6$	$\vec{K}_2 \bullet \vec{Q}_7$
$\boxed{\text{blue}} \rightarrow \vec{E}_3 \xrightarrow{W_k} \vec{K}_3$	$\vec{K}_3 \bullet \vec{Q}_1$	$\vec{K}_3 \bullet \vec{Q}_2$	$\vec{K}_3 \bullet \vec{Q}_3$	$\vec{K}_3 \bullet \vec{Q}_4$	$\vec{K}_3 \bullet \vec{Q}_5$	$\vec{K}_3 \bullet \vec{Q}_6$	$\vec{K}_3 \bullet \vec{Q}_7$
$\boxed{\text{creature}} \rightarrow \vec{E}_4 \xrightarrow{W_k} \vec{K}_4$	$\vec{K}_4 \bullet \vec{Q}_1$	$\vec{K}_4 \bullet \vec{Q}_2$	$\vec{K}_4 \bullet \vec{Q}_3$	$\vec{K}_4 \bullet \vec{Q}_4$	$\vec{K}_4 \bullet \vec{Q}_5$	$\vec{K}_4 \bullet \vec{Q}_6$	$\vec{K}_4 \bullet \vec{Q}_7$
$\boxed{\text{roamed}} \rightarrow \vec{E}_5 \xrightarrow{W_k} \vec{K}_5$	$\vec{K}_5 \bullet \vec{Q}_1$	$\vec{K}_5 \bullet \vec{Q}_2$	$\vec{K}_5 \bullet \vec{Q}_3$	$\vec{K}_5 \bullet \vec{Q}_4$	$\vec{K}_5 \bullet \vec{Q}_5$	$\vec{K}_5 \bullet \vec{Q}_6$	$\vec{K}_5 \bullet \vec{Q}_7$
$\boxed{\text{the}} \rightarrow \vec{E}_6 \xrightarrow{W_k} \vec{K}_6$	$\vec{K}_6 \bullet \vec{Q}_1$	$\vec{K}_6 \bullet \vec{Q}_2$	$\vec{K}_6 \bullet \vec{Q}_3$	$\vec{K}_6 \bullet \vec{Q}_4$	$\vec{K}_6 \bullet \vec{Q}_5$	$\vec{K}_6 \bullet \vec{Q}_6$	$\vec{K}_6 \bullet \vec{Q}_7$
$\boxed{\text{verb}} \rightarrow \vec{E}_7 \xrightarrow{W_k} \vec{K}_7$	$\vec{K}_7 \bullet \vec{Q}_1$	$\vec{K}_7 \bullet \vec{Q}_2$	$\vec{K}_7 \bullet \vec{Q}_3$	$\vec{K}_7 \bullet \vec{Q}_4$	$\vec{K}_7 \bullet \vec{Q}_5$	$\vec{K}_7 \bullet \vec{Q}_6$	$\vec{K}_7 \bullet \vec{Q}_7$

Scores $\rightarrow$ weights: why softmax?

Raw scores can be any real number. But we want a **weighted average** of the values, so the weights must be

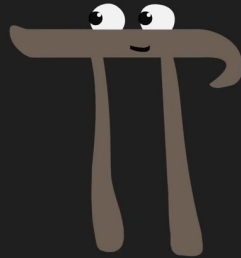
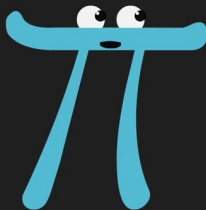
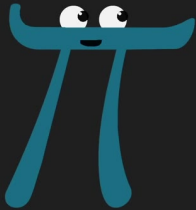
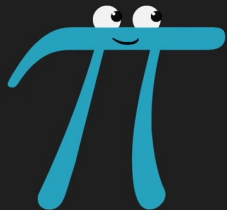
- ▶ non-negative, and
- ▶ sum to 1 (a distribution over “where to look”).

Softmax does exactly that – and *softly*: it doesn't just pick the single best key, it spreads weight over all the good ones, and it is smooth (differentiable), so we can train through it.



3Blue1Brown

	a	fluffy	blue	creature	roamed	the	verdant	forest
	$\downarrow \vec{E}_1$ $\downarrow u_0$ $\vec{Q}_1$	$\downarrow \vec{E}_2$ $\downarrow u_0$ $\vec{Q}_2$	$\downarrow \vec{E}_3$ $\downarrow u_0$ $\vec{Q}_3$	$\downarrow \vec{E}_4$ $\downarrow u_0$ $\vec{Q}_4$	$\downarrow \vec{E}_5$ $\downarrow u_0$ $\vec{Q}_5$	$\downarrow \vec{E}_6$ $\downarrow u_0$ $\vec{Q}_6$	$\downarrow \vec{E}_7$ $\downarrow u_0$ $\vec{Q}_7$	$\downarrow \vec{E}_8$ $\downarrow u_0$ $\vec{Q}_8$
a $\rightarrow \vec{E}_1 \xrightarrow{w_1} \vec{K}_1$	-0.7	-0.7	-11.7	-27.8	5.2	-89.3	-15.2	36.1
fluffy $\rightarrow \vec{E}_2 \xrightarrow{w_1} \vec{K}_2$	71.4	+2.0	5.4	+93.0	We want these to act like weights			
blue $\rightarrow \vec{E}_3 \xrightarrow{w_1} \vec{K}_3$	33.4	-5.7	+1.8	+93.4				
creature $\rightarrow \vec{E}_4 \xrightarrow{w_1} \vec{K}_4$	-21.5	-29.7	-56.1	+4.9	-82.4	-92.3	-9.5	25.1
roamed $\rightarrow \vec{E}_5 \xrightarrow{w_1} \vec{K}_5$	-20.1	-40.0	-87.8	-55.4	+9.0	-64.7	-36.1	-18.9
the $\rightarrow \vec{E}_6 \xrightarrow{w_1} \vec{K}_6$	-87.9	-33.3	-22.6	-31.4	+5.5	+0.6	-4.6	-96.8
verdant $\rightarrow \vec{E}_7 \xrightarrow{w_1} \vec{K}_7$	-41.2	-55.3	-42.3	-59.8	79.0	-97.9	-13.7	+93.8
forest $\rightarrow \vec{E}_8 \xrightarrow{w_1} \vec{K}_8$	-58.9	-75.5	-91.1	-90.6	75.6	-89.0	-70.8	-4.7



Unnormalized
Attention Pattern

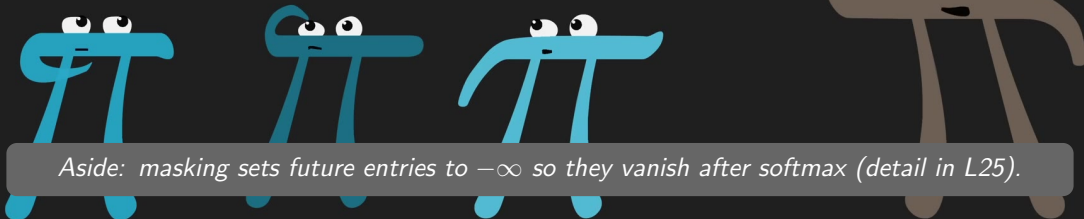
+3.53	+0.80	+1.96	+4.48	+3.74	-1.95
$-\infty$	-0.30	-0.21	+0.82	+0.29	+2.91
$-\infty$	$-\infty$	+0.89	+0.67	+2.99	-0.41
$-\infty$	$-\infty$	$-\infty$	+1.31	+1.73	-1.48
$-\infty$	$-\infty$	$-\infty$	$-\infty$	+3.07	+2.94
$-\infty$	$-\infty$	$-\infty$	$-\infty$	$-\infty$	+0.31

softmax

Normalized
Attention Pattern

1.00	0.75	0.69	0.92	0.46	0.00
0.00	0.25	0.08	0.02	0.01	0.46
0.00	0.00	0.24	0.02	0.22	0.02
0.00	0.00	0.00	0.04	0.06	0.01
0.00	0.00	0.00	0.00	0.24	0.48
0.00	0.00	0.00	0.00	0.00	0.03

This is called
“masking”



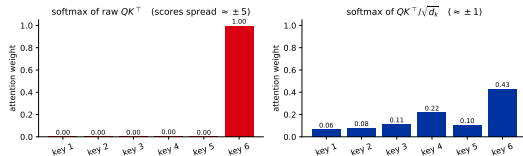
The denominator: why divide by $\sqrt{d_k}$?

A dot product in d_k dimensions is a sum of d_k terms, so its size **grows with** d_k (spread $\sim \sqrt{d_k}$). For $d_k = 64$ the raw scores swing by ± 5 or more.

Feed *big* scores to softmax and it **saturates** – one weight $\rightarrow 1$, the rest $\rightarrow 0$. That is a near-hard argmax: it looks at exactly one word, and its gradient is ≈ 0 , so **training stalls**.

Dividing by $\sqrt{d_k}$ rescales the scores back to $\approx \pm 1$, keeping softmax soft and trainable.

Why $\sqrt{d_k}$: big dimensions make dot products large $\rightarrow$ softmax collapses to a near-hard argmax ($d_k = 64$)



Putting it together: the equation

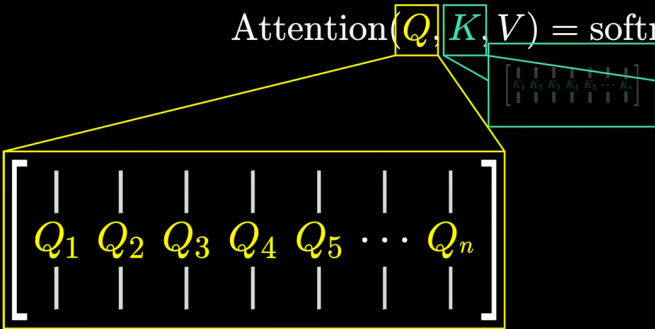
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

Read it left to right – it is exactly the three moves:

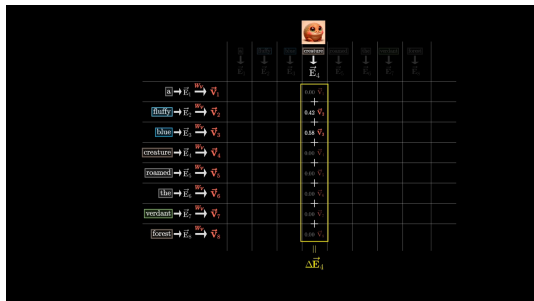
$QK^{\top}$	every query · every key = a grid of relevance scores
$\div \sqrt{d_k}$	keep the scores tame so softmax stays soft
softmax	scores → weights (positive, sum to 1), row by row
$\times V$	weighted sum of the value vectors

“Score, softmax, weighted sum” – the same three moves from the start of the deck, now with learned Q, K, V .

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{K^T Q}{\sqrt{d_k}} \right) V$$



The output: values, weighted and summed



The weights say *how much* to listen; the values say *what* to pass. The output for each word is their weighted sum:

$$\mathbf{o}_i = \sum_j \alpha_{ij} \mathbf{v}_j.$$

creature's output blends big shares of **fluffy**'s and **blue**'s values – landing on a new point in meaning-space: a *fluffy blue creature*.

Every word is updated this way **at once** – one parallel step. (In the full block this output is **added back** to the word's own vector – a residual connection – so words *refine* rather than overwrite. That's L25.)



	a $\downarrow$ $\vec{E}_1$	fluffy $\downarrow$ $\vec{E}_2$	blue $\downarrow$ $\vec{E}_3$	creature $\downarrow$ $\vec{E}_4$	roamed $\downarrow$ $\vec{E}_5$	the $\downarrow$ $\vec{E}_6$	verdant $\downarrow$ $\vec{E}_7$	forest $\downarrow$ $\vec{E}_8$	
a $\rightarrow \vec{E}_1 \xrightarrow{w_v} \vec{v}_1$				0.00 $\vec{v}_1$					
fluffy $\rightarrow \vec{E}_2 \xrightarrow{w_v} \vec{v}_2$				+ 0.42 $\vec{v}_2$					
blue $\rightarrow \vec{E}_3 \xrightarrow{w_v} \vec{v}_3$				+ 0.58 $\vec{v}_3$					
creature $\rightarrow \vec{E}_4 \xrightarrow{w_v} \vec{v}_4$				+ 0.00 $\vec{v}_1$					
roamed $\rightarrow \vec{E}_5 \xrightarrow{w_v} \vec{v}_5$				+ 0.00 $\vec{v}_5$					
the $\rightarrow \vec{E}_6 \xrightarrow{w_v} \vec{v}_6$				+ 0.00 $\vec{v}_6$					
verdant $\rightarrow \vec{E}_7 \xrightarrow{w_v} \vec{v}_7$				+ 0.00 $\vec{v}_7$					
forest $\rightarrow \vec{E}_8 \xrightarrow{w_v} \vec{v}_8$				+ 0.00 $\vec{v}_8$					

Let's run the numbers (1/2)

Three words, tiny 2-D embeddings. To keep the arithmetic visible, take $Q = K = V = X$ (skip the learned projections for a moment):

$$\mathbf{e}_{\text{fluffy}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{e}_{\text{blue}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{e}_{\text{creature}} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Scores $S = QK^T$ are all the pairwise dot products. *creature's* row:

$$\mathbf{q}_{\text{cre}} \cdot \mathbf{k}_{\text{fluffy}} = 1, \quad \mathbf{q}_{\text{cre}} \cdot \mathbf{k}_{\text{blue}} = 1, \quad \mathbf{q}_{\text{cre}} \cdot \mathbf{k}_{\text{cre}} = 2.$$

A tiny worked example ($Q = K = V = X$, 2-D embeddings)

scores $S = QK^T$ (dot products)

	fluffy	blue	creature
fluffy	1	0	1
blue	0	1	1
creature	1	1	2

weights = $\text{softmax}(S/\sqrt{d})$

	fluffy	blue	creature
fluffy	0.40	0.20	0.40
blue	0.20	0.40	0.40
creature	0.25	0.25	0.50

Let's run the numbers (2/2)

Scale *creature's* scores by $\sqrt{d} = \sqrt{2}$ and softmax:

$$\text{softmax}\left(\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}, \frac{2}{\sqrt{2}}\right) = (0.25, 0.25, 0.50).$$

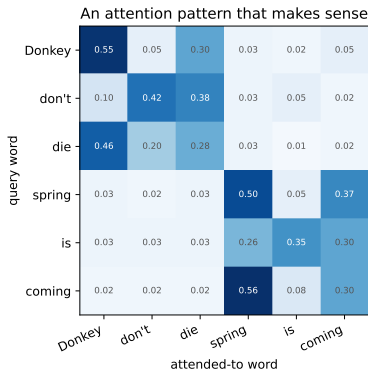
The new *creature* vector is the weighted sum of the values:

$$\mathbf{o}_{\text{cre}} = 0.25\begin{bmatrix} 1 \\ 0 \end{bmatrix} + 0.25\begin{bmatrix} 0 \\ 1 \end{bmatrix} + 0.50\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.75 \end{bmatrix}.$$

creature started at $[1, 1]$ and its attention output is $[0.75, 0.75]$ – pulled a quarter of the way toward **fluffy** and **blue**. **That is attention, by hand.** (Real models insert the learned W_Q, W_K, W_V ; the arithmetic is identical.)

A pattern that makes sense

"Donkey don't die, spring is coming" (illustrative weights)



Our sentence, with the weights laid out (row = query word, column = what it attends to):

- ▶ **coming** looks at **spring** (what is coming?),
- ▶ **die** looks at **Donkey** (who dies?),
- ▶ seasonal words cluster together.

Illustrative weights – hand-set to show the *shape* of a sensible pattern before we look at a real one.

Attention in ~ 15 lines

Every step is a matrix op. In NumPy:

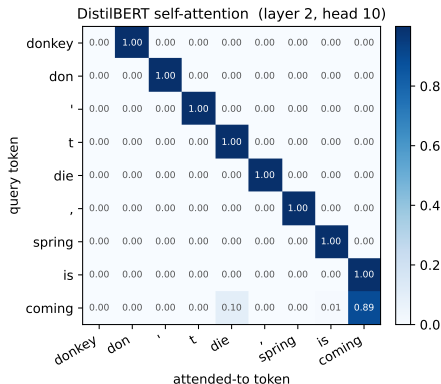
```
import numpy as np

def attention(X, Wq, Wk, Wv):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv          # three learned views
    d_k = Q.shape[1]
    scores = Q @ K.T / np.sqrt(d_k)            # relevance, scaled
    scores -= scores.max(1, keepdims=True)      # numerical stability
    weights = np.exp(scores)
    weights /= weights.sum(1, keepdims=True)    # softmax over each row
    return weights @ V                          # weighted sum of values
```

No loops over positions, no recurrence – three matmuls and a softmax. That simplicity is the whole point.

A real head, our sentence

Real model, our sentence (special tokens dropped)



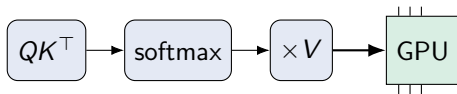
Real attention from a pretrained model (**DistilBERT**) on “*Donkey don't die, spring is coming.*” One of its heads.

This head learned a clean rule: **attend to the next token**. Nobody wrote it – the W_Q , W_K , W_V that produce it were *learned*.

Real heads specialise, and many are less tidy than our illustration. **Different heads do different jobs** – more in L25. (Special tokens dropped for clarity.)

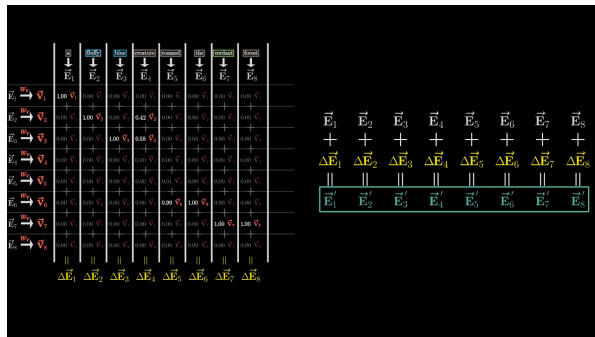
It's just matmul $\rightarrow$ it runs on a GPU

QK^T , softmax, and $\times V$ are all **big matrix multiplies** over the whole sequence at once – no step depends on the previous one.



Exactly the parallel workload a **GPU** eats (L14 “why GPUs,” L16 “a convolution is a matmul”). This is *why* attention scales – and why the RNN, which *couldn't* parallelise over time, lost.

One head of attention, end to end



- Tokens are **vectors**; directions carry meaning.
- Each word emits a **query**, a **key**, a **value** – three learned views.
- **Score** (QK^T), **scale** ($\sqrt{d_k}$), **softmax**, **weighted sum** of values.
- $\mathbf{e}' = \mathbf{e} + \Delta\mathbf{e}$: the word moves to a context-rich point.
- It's matmul $\rightarrow$ GPU $\rightarrow$ scale.

Next (L25): run many heads at once, put word order back in, and wrap it all in a repeatable block – the Transformer.

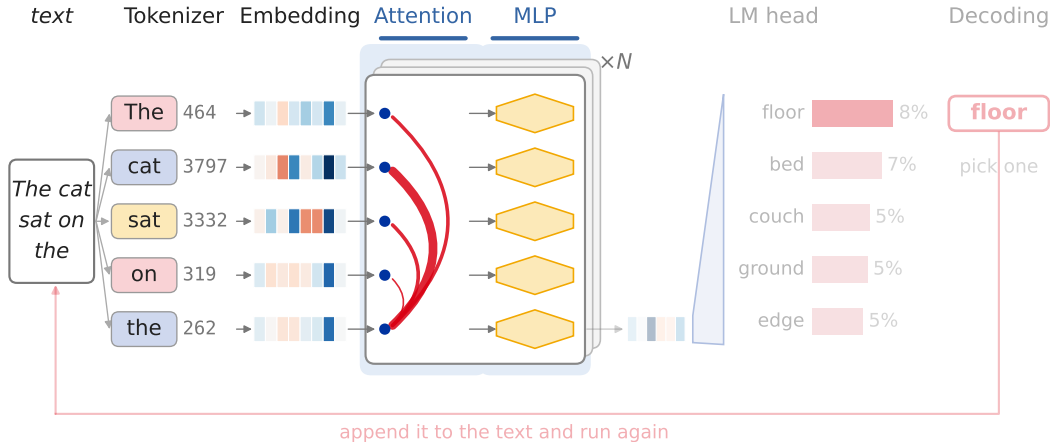
3Blue1Brown

One head of attention

	a ↓ $\vec{E}_1$	fluffy ↓ $\vec{E}_2$	blue ↓ $\vec{E}_3$	creature ↓ $\vec{E}_4$	roamed ↓ $\vec{E}_5$	the ↓ $\vec{E}_6$	verdant ↓ $\vec{E}_7$	forest ↓ $\vec{E}_8$
$\vec{E}_1 \xrightarrow{w_v} \vec{v}_1$	1.00 $\vec{v}_1$	0.00 $\vec{v}_1$	0.00 $\vec{v}_1$	0.00 $\vec{v}_1$	0.00 $\vec{v}_1$	0.00 $\vec{v}_1$	0.00 $\vec{v}_1$	0.00 $\vec{v}_1$
$\vec{E}_2 \xrightarrow{w_v} \vec{v}_2$	0.00 $\vec{v}_2$	1.00 $\vec{v}_2$	0.00 $\vec{v}_2$	0.42 $\vec{v}_2$	0.00 $\vec{v}_2$	0.00 $\vec{v}_2$	0.00 $\vec{v}_2$	0.00 $\vec{v}_2$
$\vec{E}_3 \xrightarrow{w_v} \vec{v}_3$	0.00 $\vec{v}_3$	0.00 $\vec{v}_3$	1.00 $\vec{v}_3$	0.58 $\vec{v}_3$	0.00 $\vec{v}_3$	0.00 $\vec{v}_3$	0.00 $\vec{v}_3$	0.00 $\vec{v}_3$
$\vec{E}_4 \xrightarrow{w_v} \vec{v}_4$	0.00 $\vec{v}_4$	0.00 $\vec{v}_4$	0.00 $\vec{v}_4$	0.00 $\vec{v}_4$	0.00 $\vec{v}_4$	0.00 $\vec{v}_4$	0.00 $\vec{v}_4$	0.00 $\vec{v}_4$
$\vec{E}_5 \xrightarrow{w_v} \vec{v}_5$	0.00 $\vec{v}_5$	0.00 $\vec{v}_5$	0.00 $\vec{v}_5$	0.00 $\vec{v}_5$	0.01 $\vec{v}_5$	0.00 $\vec{v}_5$	0.00 $\vec{v}_5$	0.00 $\vec{v}_5$
$\vec{E}_6 \xrightarrow{w_v} \vec{v}_6$	0.00 $\vec{v}_6$	0.00 $\vec{v}_6$	0.00 $\vec{v}_6$	0.00 $\vec{v}_6$	0.99 $\vec{v}_6$	1.00 $\vec{v}_6$	0.00 $\vec{v}_6$	0.00 $\vec{v}_6$
$\vec{E}_7 \xrightarrow{w_v} \vec{v}_7$	0.00 $\vec{v}_7$	0.00 $\vec{v}_7$	0.00 $\vec{v}_7$	0.00 $\vec{v}_7$	0.00 $\vec{v}_7$	1.00 $\vec{v}_7$	1.00 $\vec{v}_7$	0.00 $\vec{v}_7$
$\vec{E}_8 \xrightarrow{w_v} \vec{v}_8$	0.00 $\vec{v}_8$	0.00 $\vec{v}_8$	0.00 $\vec{v}_8$	0.00 $\vec{v}_8$	0.00 $\vec{v}_8$	0.00 $\vec{v}_8$	0.00 $\vec{v}_8$	0.00 $\vec{v}_8$
	$\Delta \vec{E}_1$	$\Delta \vec{E}_2$	$\Delta \vec{E}_3$	$\Delta \vec{E}_4$	$\Delta \vec{E}_5$	$\Delta \vec{E}_6$	$\Delta \vec{E}_7$	$\Delta \vec{E}_8$

$\vec{E}_1$	$\vec{E}_2$	$\vec{E}_3$	$\vec{E}_4$	$\vec{E}_5$	$\vec{E}_6$	$\vec{E}_7$	$\vec{E}_8$
+	+	+	+	+	+	+	+
$\Delta \vec{E}_1$	$\Delta \vec{E}_2$	$\Delta \vec{E}_3$	$\Delta \vec{E}_4$	$\Delta \vec{E}_5$	$\Delta \vec{E}_6$	$\Delta \vec{E}_7$	$\Delta \vec{E}_8$
$\vec{E}'_1$	$\vec{E}'_2$	$\vec{E}'_3$	$\vec{E}'_4$	$\vec{E}'_5$	$\vec{E}'_6$	$\vec{E}'_7$	$\vec{E}'_8$

Next on the map



Next (L25): many heads at once, word order put back in, and the MLP - one layer, repeated N times.

Sources & further reading

- ▶ Bahdanau, Cho & Bengio (2015), *Neural Machine Translation by Jointly Learning to Align and Translate* – attention, for translation.
- ▶ Xu et al. (2015), *Show, Attend and Tell* – attention for image captioning.
- ▶ Vaswani et al. (2017), *Attention Is All You Need* – self-attention, the Transformer.
- ▶ Mikolov et al. (2013), word2vec – word embeddings and analogies.
- ▶ Jay Alammar, *The Illustrated Transformer*, Andrej Karpathy, “Let’s build GPT.”
- ▶ **3Blue1Brown**, “Deep Learning” Ch. 5–6 – the visual intuition this deck leans on.

For the full, animated story of attention, watch 3Blue1Brown Ch. 6 – it is the clearest explanation out there.